

CS301 Database Systems — Final Examination Master Solution Compendium

Fall 2025 | CONFIDENTIAL — Faculty & Teaching Assistants Only

CONFIDENTIAL — FACULTY & TEACHING ASSISTANT USE ONLY. This CS301 Database Management Systems examination solution compendium is restricted to the Course Instructor of Record and certified teaching assistants. Unauthorized disclosure to students prior to official result publication constitutes a Level 3 Academic Integrity Policy Violation under Section 6.3 of the University Disciplinary Matrix.

1. Examination Administrative Overview

Course: CS301 Database Management Systems | Term: Fall 2025 | Duration: 3 Hours Total Marks: 100 | Passing Floor: 50 | Lead Examiner: Dr. Elena Marsh, Associate Professor. Evaluator Guidance: Award marks based on conceptual clarity, formal correctness, and sound architectural reasoning. Alternate valid formalisms and SQL syntaxes yielding equivalent logical execution receive full credit. Deduct 20% of section marks for each major conceptual error. Deduct 10% for each minor computational error with otherwise correct method.

2. Unit I: ER Modeling & Relational Schema Design (Q1 — 20 Marks)

Q1.1: Entity vs Relationship; Weak Entity Mapping — 10 Marks

Model Answer: An **Entity** represents a distinguishable real-world object or concept with independent existence, characterized by descriptive attributes — e.g., Student (StudentID, Name, Programme), Course (CourseCode, Title, Credits), Department (DeptID, Name). A **Relationship** captures a semantic association among entity instances — e.g., Student ENROLLS_IN Course (with attributes: Grade, EnrollmentDate), Professor ADVISES Student. Relationships may carry their own attributes when the attribute belongs to the association rather than either participating entity.

A **Weak Entity Set** lacks sufficient own attributes to form a primary key; its existence depends entirely on an *identifying (owner) entity set* via an *identifying relationship*. Key example: Dependent (Name, DateOfBirth, Relationship) exists only in relation to Employee — the discriminator 'Dependent_Name' alone cannot uniquely identify a Dependent across all employees.

Relational Mapping of Weak Entities (7-step algorithm, Step 5): (1) Create a table for the weak entity with all its attributes. (2) Include the complete Primary Key of the owner entity as a Foreign Key. (3) The Primary Key of the resulting table is the composite {Owner_PK + Discriminator_Attribute}. (4) Define the FK constraint with ON DELETE CASCADE to enforce total existential dependency.

Marking: 4M (entity vs relationship distinction with example); 3M (weak entity, identifying relationship, discriminator concept); 3M (exact relational mapping with composite PK and ON DELETE CASCADE).

Q1.2: Specialization, Generalization, and Aggregation — 10 Marks

Model Answer:

- **Specialization (Top-down):** Defining sub-groups within a higher-level entity set based on distinguishing characteristics. E.g., Employee specializes into Faculty (Tenure_Status, ResearchArea) and AdministrativeStaff (Department_Assigned, Grade_Level). Implemented relationally by creating separate tables for each sub-entity containing only subtype-specific attributes plus the supertype PK as FK.

- **Generalization (Bottom-up):** Synthesizing multiple entity sets sharing common attributes into a single higher-level superclass. E.g., Car and Motorcycle are generalized into Vehicle (VehicleID, VIN, ManufacturingYear, OwnerID). Disjoint vs overlapping constraints; total vs partial participation constraints determine whether every superclass instance must belong to a subclass.

- **Aggregation:** Treats a relationship and its participating entities as a composite higher-level entity, enabling a relationship to participate in another relationship. E.g., the 'Employee WORKS_ON Project' relationship is aggregated as a unit, then associated with 'Manager SUPERVISES {Employee-WORKS_ON-Project}'. This models supervision of a work assignment rather than supervision of an employee globally.

Marking: 3M (specialization with mapping rule); 3M (generalization with disjoint/overlapping distinction); 4M (aggregation with illustrative diagram and relational mapping).

3. Unit II & III: Normalization Theory & Functional Dependencies (Q2 — 30 Marks)

Q2.1: Armstrong's Axioms — Transitivity Proof — 10 Marks

Model Answer: A Functional Dependency $X \rightarrow Y$ holds on relation schema R if for all legal instances r of R, $\forall t1, t2 \in r: t1[X] = t2[X] \Rightarrow t1[Y] = t2[Y]$. X functionally determines Y; Y is functionally dependent on X.

Armstrong's Axioms (Sound and Complete):

A1 — Reflexivity: If $Y \subseteq X$, then $X \rightarrow Y$ (trivial dependency).

A2 — Augmentation: If $X \rightarrow Y$, then $XZ \rightarrow YZ$ for any attribute set Z.

A3 — Transitivity: If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$.

Derived Rules: Union ($X \rightarrow Y, X \rightarrow Z \Rightarrow X \rightarrow YZ$), Decomposition ($X \rightarrow YZ \Rightarrow X \rightarrow Y, X \rightarrow Z$), Pseudotransitivity ($X \rightarrow Y, WY \rightarrow Z \Rightarrow WX \rightarrow Z$).

Formal Proof of Transitivity:

Given: $X \rightarrow Y$ (1) and $Y \rightarrow Z$ (2). Prove: $X \rightarrow Z$.

Step 1: From (1) by A2 (augment with X): $XX \rightarrow XY \Rightarrow X \rightarrow XY$.

Step 2: From (2) by A2 (augment with X): $XY \rightarrow XZ$.

Step 3: From Steps 1 and 2, since $X \rightarrow XY$ and $XY \rightarrow XZ$, by A3: $X \rightarrow XZ$.

Step 4: By A1 (reflexivity): $XZ \rightarrow Z$ (since $Z \subseteq XZ$).

Step 5: By A3 from Steps 3 and 4: $X \rightarrow Z$. ■ Q.E.D.

Marking: 3M (FD formal definition); 4M (three axioms with derived rules); 3M (formal proof).

Q2.2: Candidate Keys, Normal Form Determination, and 3NF Decomposition — 20 Marks

Given: R(A, B, C, D, E) with F = { $A \rightarrow BC, CD \rightarrow E, B \rightarrow D, E \rightarrow A$ }.

(i) All Candidate Keys — 6M:

Strategy: Compute attribute closures to find all minimal superkeys.

• $A^\# = \{A\} \rightarrow \{A, B, C\}$ (via $A \rightarrow BC$) $\rightarrow \{A, B, C, D\}$ (via $B \rightarrow D$) $\rightarrow \{A, B, C, D, E\}$ (via $CD \rightarrow E$). $A^\# = ABCDE \rightarrow A$ is a candidate key.

• $E^\# = \{E, A, B, C, D, E\}$ (via $E \rightarrow A$, then $A \rightarrow BC, B \rightarrow D, CD \rightarrow E$) = $ABCDE \rightarrow E$ is a candidate key.

• $B^\# = \{B, D\}$ (via $B \rightarrow D$). $B^\# \neq ABCDE \rightarrow B$ is NOT a CK.

• $BC^\# = \{B, C, D\} \rightarrow \{B, C, D, E\}$ (via $CD \rightarrow E$) $\rightarrow \{A, B, C, D, E\}$ (via $E \rightarrow A$) = $ABCDE \rightarrow BC$ is a candidate key.

• $CD^\# = \{C, D, E\}$ (via $CD \rightarrow E$) $\rightarrow \{A, B, C, D, E\}$ (via $E \rightarrow A, A \rightarrow BC$) = $ABCDE \rightarrow CD$ is a candidate key.

Prime attributes = {A, B, C, D, E} (every attribute appears in at least one CK). Non-prime attributes = {} (empty set).

(ii) Highest Normal Form — 6M:

BCNF requires: for every non-trivial FD $X \rightarrow Y$ in F, X must be a superkey.

• $A \rightarrow BC$: A is a CK $\rightarrow$ satisfies BCNF. ✓

• $CD \rightarrow E$: CD is a CK $\rightarrow$ satisfies BCNF. ✓

• $B \rightarrow D$: B is NOT a superkey $\rightarrow$ VIOLATES BCNF. However, D is a prime attribute ($D \in CK = \{CD\}$). Since D is prime, $B \rightarrow D$ satisfies 3NF (3NF condition: either X is a superkey, or $Y - X$ contains only prime attributes). ✓ 3NF.

• $E \rightarrow A$: E is a CK $\rightarrow$ satisfies BCNF. ✓

Conclusion: R is in 3NF but NOT in BCNF due to the BCNF violation of $B \rightarrow D$.

(iii) 3NF Decomposition via Bernstein's Synthesis — 8M:

Step 1: Compute canonical cover F_c :

$A \rightarrow BC$ decomposes to $A \rightarrow B, A \rightarrow C$ (no extraneous attributes since B and C are independent).

$F_c = \{A \rightarrow B, A \rightarrow C, B \rightarrow D, CD \rightarrow E, E \rightarrow A\}$.

Step 2: Create one relation for each FD in F_c :

R1(A,B) with PK=A; R2(A,C) with PK=A; R3(B,D) with PK=B; R4(C,D,E) with PK={C,D}; R5(E,A) with PK=E.

Step 3: Merge relations with same key: R1(A,B) and R2(A,C) merge to R12(A,B,C) PK=A.

Step 4: Final decomposition: {R12(A,B,C), R3(B,D), R4(C,D,E), R5(E,A)}.

Step 5: Verify candidate key containment: R12 contains CK A ✓, R4 contains CK CD ✓, R5 contains CK E ✓ $\rightarrow$ no additional universal-key relation needed.

Proof of Lossless-Join: The join on R12 ■ R3 ■ R4 ■ R5 reconstructs R without spurious tuples (verified by tableau method). Proof of Dependency-Preservation: All FDs in F_c are preserved in individual relations. Both properties satisfied. ■

4. Unit IV: Transactions, ACID & Concurrency (Q3 — 30 Marks)

Q3.1: ACID Properties & Write-Ahead Logging (WAL) / ARIES Recovery — 15 Marks

Model Answer:

- **Atomicity:** All operations of a transaction execute to completion, or none take effect. Ensures no partial updates reach the database. Enforced via undo-logging (ROLLBACK on abort).
- **Consistency:** A transaction transforms the database from one integrity-valid state to another. All integrity constraints, foreign keys, domain constraints, and application-level invariants must hold before and after each transaction.
- **Isolation:** Concurrent transactions execute as if they were serial. Each transaction sees a consistent snapshot. Enforced via lock managers (2PL) or MVCC. Isolation levels: Read Uncommitted, Read Committed, Repeatable Read, Serializable (ANSI SQL).
- **Durability:** Once a transaction commits, its effects persist permanently through any subsequent system failure. Enforced via redo-logging and stable (non-volatile) storage.

Write-Ahead Logging (WAL) — Core Rules:

Rule 1 — Write-Ahead Rule: Before any modified data page is written from the buffer pool to disk, all log records for that page (including undo information) must first be flushed to stable storage (the WAL log file).

Rule 2 — Commit Rule: A transaction is not declared committed until its COMMIT log record has been force-flushed to stable storage.

ARIES Recovery Algorithm (3 Phases):

Phase 1 — Analysis Pass: Scan the log forward from the last stable checkpoint. Reconstruct the dirty-page table (DPT — pages modified but not yet written to disk) and the active-transaction table (ATT — transactions not yet committed at crash time). Determine the RedoLSN (earliest log record from which redo must begin).

Phase 2 — Redo Pass: Replay all logged operations from RedoLSN forward, bringing the database to the exact state it was in at the moment of crash. Redo is applied even to already-committed transactions to handle cases where buffer pages had not been flushed.

Phase 3 — Undo Pass: Roll back all transactions listed as active (uncommitted) at crash time in reverse chronological order using the CLR (Compensation Log Records) mechanism to ensure idempotent recovery.

Q3.2: Strict 2PL — Conflict Serializability Proof & Cascading Abort Prevention — 15 Marks

Two-Phase Locking (2PL): Standard 2PL restricts each transaction to two distinct phases: (1) Growing Phase: transaction acquires locks (shared S or exclusive X) as needed. (2) Shrinking Phase: transaction only releases locks; no new locks may be acquired once the first lock is released. The point of first lock release is the transaction's *lock point*.

Strict 2PL: All exclusive (X) locks are held until after the transaction commits or aborts, preventing dirty reads.

Proof of Conflict Serializability: Assume for contradiction that a schedule S produced by strict 2PL is NOT conflict-serializable. Then S has a cycle $T_1 \rightarrow T_2 \rightarrow \dots \rightarrow T_n \rightarrow T_1$ in its precedence (serialization) graph. An edge $T_i \rightarrow T_{i+1}$ implies T_i performed an operation on data item X before T_{i+1} performed a conflicting operation on X, requiring T_i to have released its lock on X before T_{i+1} acquired it.

Therefore, $\text{lock_point}(T_i) < \text{lock_point}(T_{i+1})$. For the cycle $T_1 \rightarrow T_2 \rightarrow \dots \rightarrow T_n \rightarrow T_1$, this requires $\text{lock_point}(T_1) < \text{lock_point}(T_2) < \dots < \text{lock_point}(T_n) < \text{lock_point}(T_1)$, which is a contradiction (a real number cannot be strictly less than itself). Therefore, no such cycle can exist; every strict 2PL schedule is conflict-serializable. ■

Prevention of Cascading Aborts: A cascading rollback occurs when T_i writes X, T_j reads T_i 's uncommitted write, T_i aborts $\rightarrow T_j$ must also abort, potentially propagating to T_k , etc. Under Strict 2PL, T_i retains its X-lock on X until commit/abort. Therefore, T_j cannot acquire even a shared read lock on X while T_i holds the X-lock. Dirty reads are structurally prevented, and cascading aborts are entirely eliminated.

5. Unit V: Indexing & Query Optimization (Q4 — 20 Marks)

Q4.1: B-Tree vs B+ Tree; Superiority for Disk-Based Storage — 10 Marks

B-Tree: In a B-Tree of order m , both internal (index) nodes and leaf nodes store search key values AND pointers to actual data records. This means data can be found at any level of the tree during a search traversal.

B+ Tree: Internal nodes store only search keys and child pointers — acting as a pure navigation/routing layer. All actual data pointers (or full data tuples in a clustered index) are stored exclusively in the leaf nodes. All leaf nodes are linked together in a sorted doubly-linked list (leaf sibling pointers).

Why B+ Trees Dominate Relational Database Indexes:

1. **Higher Fan-out → Shallower Tree Height:** Since internal nodes contain only keys (no data pointers), more keys fit in one disk page (page size typically 8KB or 16KB). Higher branching factor reduces tree height. A B+ tree of order 200 can index $200^3 = 8$ million records with height 3, requiring only 3 disk I/Os for point lookups.
2. **Superior Range Query Performance:** B-Tree range queries require complex in-order traversal across multiple tree levels. B+ Tree range queries locate the starting leaf node and then traverse the linked list horizontally across leaf nodes — efficient, sequential disk reads ideal for buffer prefetching.
3. **Uniform Query Latency:** Every key lookup traverses exactly root-to-leaf depth (same number of disk I/Os), enabling predictable query performance without worst-case variability.
4. **Clustered Index Implementation:** Since leaves store full tuples (or row pointers) in sorted key order, the B+ Tree naturally supports clustered index access — records with similar key values are physically collocated on disk, minimizing random I/O for range scans.

Q4.2: Heuristic Query Optimization & Equivalence Rules — 10 Marks

Query Optimization Goal: Rewrite a logically correct query into an equivalent but computationally cheaper execution plan, reducing disk I/O, memory consumption, and CPU cycles.

Heuristic Rules (Algebraic Equivalence Transformations):

Rule 1 — **Selection Push-Down (Cascade):** Push selection operations as early as possible in the query tree, applying them immediately after the table scan. Reduces intermediate relation cardinality before expensive join operations. E.g., $\sigma_{\text{Dept}='CS'}(\text{Employee} \bowtie \text{Dept}) \rightarrow \sigma_{\text{Dept}='CS'}(\text{Employee}) \bowtie \text{Dept}$.

Rule 2 — **Projection Push-Down:** Push projection operations down to remove unnecessary columns early, reducing tuple byte width. Fewer bytes per tuple $\rightarrow$ more tuples fit in one buffer block $\rightarrow$ fewer disk I/Os for subsequent operations.

Rule 3 — **Combining Cascaded Unary Operations:** Merge consecutive selections $(\sigma_{C1}(\sigma_{C2}(R))) = \sigma_{C1 \wedge C2}(R)$ and combine adjacent selections with projections into single passes over the relation.

Rule 4 — **Join Ordering by Selectivity:** Execute the most selective joins first (those producing the smallest intermediate result) to minimize the size of data flowing into subsequent join stages. Dynamic programming over all join orderings gives the globally optimal plan (used in PostgreSQL's planner for $N \leq 8$ relations; Genetic Query Optimizer (GEQO) for $N > 8$).

Rule 5 — **Replace Cartesian Products with Joins:** Wherever a Cartesian product ($\times$) is followed by a selection condition linking the two relations, replace them with the corresponding join operator. Cartesian products produce $|R| \times |S|$ tuples before filtering; joins apply the predicate during the merge step.

Implementation Techniques: Index Nested Loop Join (use existing B+ tree index on join attribute), Sort-Merge Join (for sorted or sortable inputs), Hash Join (for large unsorted inputs with equality join predicates).

6. Marking Summary & Grade Boundary Reference

Question-wise Mark Distribution: Q1 (20M): Q1.1=10, Q1.2=10. Q2 (30M): Q2.1=10, Q2.2=20. Q3 (30M): Q3.1=15, Q3.2=15. Q4 (20M): Q4.1=10, Q4.2=10. Total: 100M. Grade boundaries: O (Outstanding) ≥ 90 ; A+ (Excellent) ≥ 80 ; A (Very Good) ≥ 70 ; B+ (Good) ≥ 60 ; B (Above Average) ≥ 55 ; C (Pass) ≥ 50 ; F (Fail) < 50 .